

Prova - Introdução ao R

Pedro Vitor Carvalho Ferreira da Silva

2026/09/25

Escolha da semente

Para a realização da atividade a semente escolhida foi o número 81:

```
set.seed(81)
```

Questão 1 - Tipos de dados e fatores (2 pontos)

a) Explique, com suas palavras, a diferença entre os tipos `character` e `factor` em R.

A diferença principal entre os tipos `character` (`chr`) e `factor` é que o `chr` representa um vetor com `n` strings diferentes, enquanto o `factor` agrupa strings iguais em níveis (`levels`) enumerados, tornando mais fácil manipulação de dados.

b) Crie o vetor abaixo e indique sua classe com um comando de R.

```
v <- c("macho", "fêmea", "macho", "fêmea")
```

Para retornar a classe foi usada a função `class()`:

```
class(v)
```

E foi retornada a classe `character`:

```
## [1] "character"
```

c) Converta `v` em fator, definindo os níveis nesta ordem: “fêmea”, “macho”.

Para converter `v` em fator foi usada a função `factor()` e o argumento `levels`:

```
f <- factor(v, levels = c("fêmea", "macho"))
```

d) Mostre a saída de `levels()` e explique o que mudou. Os valores observados mudaram?

Para retornar os `levels` foi usada a função `levels()`:

```
levels(f)
```

E foi retornado:

```
## [1] "fêmea" "macho"
```

Nesse caso não houve diferença entre definir os níveis com argumentos da função, pois o R ordena os níveis por ordem alfabética. Logo, “fêmea” é representado pelo número 1 e “macho” pelo número 2.

Questão 2 - Vetores, indexação e matrizes (2 pontos)

a) Crie um vetor x com 10 inteiros aleatórios distintos entre 1 e 100.

Para criar esse vetor foi utilizada a função `sample()` com o argumento `replace` definido como falso para evitar repetição dos números:

```
x = sample(1:100, 10, replace = FALSE)
```

b) Extraia os elementos das posições 3 a 7 e, separadamente, os das posições ímpares (1, 3, 5, 7 e 9).

Para extrair os argumentos das posições 3 a 7:

```
x[3:7]
```

E foi retornado:

```
## [1] 86 12 81 10 9
```

Já para extrair apenas dos endereços ímpares:

```
x[c(1, 3, 5, 7, 9)]
```

E foi retornado:

```
## [1] 99 86 81 9 85
```

Durante a realização da prova descobri um método mais prático e maleável de extrair apenas os ímpares:

```
x[c(T, F)]
```

```
## [1] 99 86 81 9 85
```

c) Transforme x em uma matriz m de 2 linhas por 5 colunas.

Para converter x em matriz foi usada a função `matrix()` com os argumentos `nrow` e `ncol`:

```
m <- matrix(x, nrow = 2, ncol = 5)
```

d) Extraia a primeira linha e a última coluna de m.

Para extrair a primeira linha:

```
m[1,]
```

E foi retornado:

```
## [1] 99 86 81 9 85
```

Já para extrair a última coluna:

```
m[,ncol(m)]
```

E foi retornado:

```
## [1] 85 37
```

e) Explique o resultado de `m[2, ]` e como `matrix()` distribuiu os elementos de `x` na matriz, por padrão.

O resultado de `m[2, ]`:

```
## [1] 25 12 10 54 37
```

O padrão do R é distribuir os elementos de um vetor na matriz preenchendo por colunas. Isso significa que numa matriz de 2 linhas os dois primeiros valores estarão na primeira coluna, o terceiro e o quarto valor estarão na segunda coluna e assim por diante. Para alterar isso pode ser utilizado o argumento `byrow`, definindo como falso, e assim fazendo com que os valores sejam preenchidos por linhas.

Questão 3 - Data frame, subconjuntos e operadores lógicos (2 pontos)

Crie um `data.frame` chamado `dados` com 6 linhas e estas colunas:

- **Sexo:** valores “M” e “F”, incluindo pelo menos um de cada;
- **Peso:** 6 valores de `rnorm()` com média 20;
- **Altura:** 6 valores de `rnorm()` com média 10.

Mostre dados e responda:

Para criar variáveis e o `data.frame`:

```
s <- c("M", "F", "F", "M", "F", "M")
p <- rnorm(6, mean = 20)
a <- rnorm(6, mean = 10)
dados <- data.frame(Sexo = s, Peso = p, Altura = a)
```

Resultado do data.frame:

##	Sexo	Peso	Altura
## 1	M	19.77696	10.968675
## 2	F	18.60745	10.507864
## 3	F	19.65212	8.635634
## 4	M	19.30043	10.441194
## 5	F	21.05741	8.585636
## 6	M	19.91101	9.766473

a) Use `subset()` para selecionar indivíduos com Peso maior que a média de todos os pesos em dados.

Para achar a média usada a função `mean()` dentro da função `subset()`:

```
subset(dados, Peso > mean(dados$Peso))
```

E foi retornado:

##	Sexo	Peso	Altura
## 1	M	19.77696	10.968675
## 5	F	21.05741	8.585636
## 6	M	19.91101	9.766473

b) Selecione indivíduos do sexo feminino com Altura menor que a mediana de todas as alturas em dados.

Para achar a mediana usada a função `median()` dentro da função `subset()`:

```
subset(dados, Sexo == "F" & Altura < median(dados$Altura))
```

E foi retornado:

##	Sexo	Peso	Altura
## 3	F	19.65212	8.635634
## 5	F	21.05741	8.585636

c) Selecione indivíduos com `Peso > 20` ou `Sexo == "M"`.

O comando executado:

```
subset(dados, Peso > 20 | Sexo == "M")
```

E foi retornado:

##	Sexo	Peso	Altura
## 1	M	19.77696	10.968675
## 4	M	19.30043	10.441194
## 5	F	21.05741	8.585636
## 6	M	19.91101	9.766473

d) Explique a diferença entre & e | nas condições acima.

O símbolo & (“and”) serve para que apenas sejam amostrados dados em que duas condições sejam verdadeiras, enquanto o símbolo | (“or”) amostra quando pelo menos uma das condições é atendida.

Questão 4 - Fatores, boxplot e interpretação (2 pontos)

a) Crie um vetor mes que contenha os 12 meses do ano, cada um aparecendo três vezes (36 observações).

Para criar as repetições de forma eficiente foi usada a função `rep()`:

```
mes <- c(
  rep("Janeiro", 3),
  rep("Fevereiro", 3),
  rep("Março", 3),
  rep("Abril", 3),
  rep("Maio", 3),
  rep("Junho", 3),
  rep("Julho", 3),
  rep("Agosto", 3),
  rep("Setembro", 3),
  rep("Outubro", 3),
  rep("Novembro", 3),
  rep("Dezembro", 3)
)
```

b) Gere um vetor peso com 36 inteiros aleatórios entre 1 e 100.

Código executado para gerar o vetor:

```
peso = sample(1:100, 36)
```

c) Crie um data.frame com as colunas Mês (fator com níveis em ordem cronológica) e Peso.

Para criar o data.frame:

```
data <- data.frame(Mês = mes, Peso = peso)
```

Para transformar em fator e organizar em ordem cronológica:

```
data$Mês <- factor(
  data$Mês,
  levels = c(
    "Janeiro",
    "Fevereiro",
    "Março",
    "Abril",
    "Maio",
    "Junho",
    "Julho",
    "Agosto",
    "Setembro",
    "Outubro",
    "Novembro",
    "Dezembro"
  )
)
```

```

    "Julho",
    "Agosto",
    "Setembro",
    "Outubro",
    "Novembro",
    "Dezembro"
  )
)

```

Resultado do data.frame:

```

##           Mês  Peso
## 1   Janeiro   54
## 2   Janeiro   87
## 3   Janeiro   68
## 4 Fevereiro   14
## 5 Fevereiro   28
## 6 Fevereiro   10
## 7     Março   84
## 8     Março   91
## 9     Março   63
## 10    Abril   82
## 11    Abril   25
## 12    Abril    3
## 13     Maio   90
## 14     Maio   59
## 15     Maio   34
## 16     Junho   78
## 17     Junho   62
## 18     Junho   56
## 19     Julho   39
## 20     Julho   83
## 21     Julho   46
## 22    Agosto   48
## 23    Agosto   16
## 24    Agosto   96
## 25 Setembro   86
## 26 Setembro   11
## 27 Setembro   95
## 28 Outubro   18
## 29 Outubro   98
## 30 Outubro   97
## 31 Novembro   44
## 32 Novembro   67
## 33 Novembro   40
## 34 Dezembro   31
## 35 Dezembro   22
## 36 Dezembro   47

```

d) Faça um boxplot de Peso por Mês, com título e nomes dos eixos.

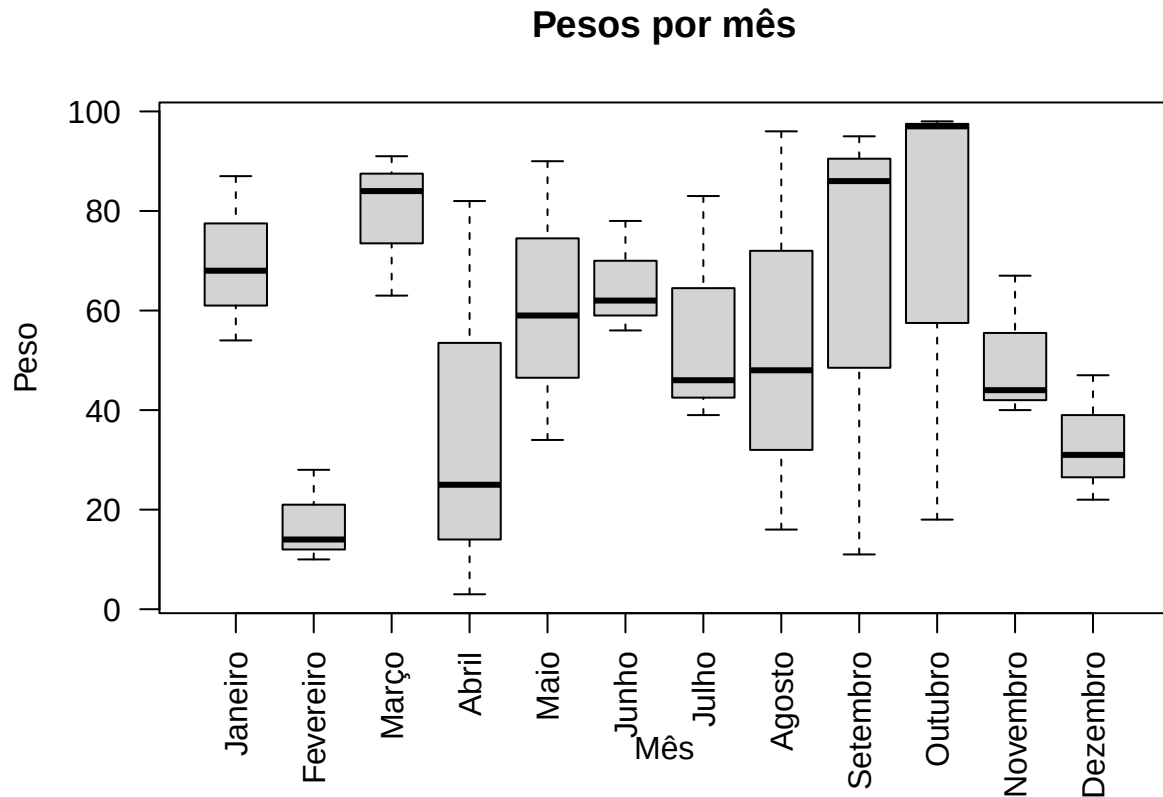
Para que os nomes dos eixos pudessem ser acessados sem mencionar o data.frame foi usada a função `attach()`:

```
attach(data)
```

Assim foi feito o boxplot com nomes dos eixos, título e alinhamento vertical dos meses:

```
boxplot(Peso~Mês, main = "Pesos por mês", las = 2)
```

E esse foi o resultado:



e) Explique: o que representa a caixa? O que representa a linha dentro dela? Até onde vão os bigodes? O que representam os pontos fora dos bigodes? Como comparar a dispersão entre meses? Não interprete esses dados simulados como evidência ecológica.

A caixa representa o intervalo entre o primeiro quartil e o terceiro quartil contendo, portanto, os 50% centrais dos dados, enquanto a linha dentro da caixa representa a mediana. Os bigodes se estendem até os valores mais extremos que ainda estão dentro de 150% do intervalo da caixa, assim não representando necessariamente os valores mínimo e máximo. Já os pontos fora dos bigodes, caso estivessem presentes nesse bloxplot, representariam valores considerados potenciais outliers, pois estariam além dos limites definidos pelos bigodes. A dispersão entre meses pode ser comparada principalmente pelo tamanho das caixas, considerando uma caixa de maior tamanho como indicador de maior dispersão nos 50% centrais dos valores. Ademais, o comprimento dos bigodes e a quantidade e/ou distância dos pontos fora deles também podem ser considerados.

Questão 5 - Manipulação de dados e ordenação (2 pontos)

Crie o objeto abaixo. Você pode definir uma semente antes do comando.

```
dados2 <- data.frame(  
  Sexo = c("M", "F", "F", "M", "M", "F"),  
  Local = c("1", "1", "1", "2", "2", "2"),  
  Peso = rnorm(6, mean = 20)  
)
```

OBS.: A semente já foi previamente definida no início do documento (81)

a) Crie a coluna SL com paste(Sexo, Local).

O comandos executados para criação e adição da coluna:

```
S_L <- paste(dados2$Sexo, dados2$Local)  
dados2 <- cbind(dados2, S_L)
```

b) Atribua às linhas os nomes p1 a p6.

Para atribuir nomes às linhas foi utilizada a função rownames():

```
rownames(dados2) <- c("p1", "p2", "p3", "p4", "p5", "p6")
```

c) Renomeie as colunas, nesta ordem, para Sex, Loc, P e SL.

Para renomear as colunas foi utilizada a função colnames():

```
colnames(dados2) <- c("Sex", "Loc", "P", "SL")
```

d) Ordene as linhas por SL em ordem crescente e, em caso de empate, por P em ordem decrescente.

Para ordenar as linhas foi utilizada a função order():

```
dados2 <- dados2[order(dados2$SL, -dados2$P), ]
```

E, por fim, assim ficou o data.frame:

```
##      Sex Loc      P  SL  
## p2    F   1 20.04689 F 1  
## p3    F   1 18.72910 F 1  
## p6    F   2 20.24890 F 2  
## p1    M   1 19.84907 M 1  
## p5    M   2 21.65471 M 2  
## p4    M   2 19.96807 M 2
```

e) Explique o papel de `-dados2$P` dentro de `order()` e por que isso funciona neste exemplo.

O sinal - (menos) antes de `dados2$P` permite que os valores numéricos da coluna P sejam ordenados em ordem decrescente, diferentemente do padrão da função `order()`, a qual organiza os valores numéricos em ordem crescente. Isso apenas é possível por conta da composição da coluna, por valores numéricos.